

Análise de Integração VokeNext → Sankhya

Documentação Funcional e Técnica — US 1085 "Consultar chamados do VokeNex via API para integração com o Sankhya"

Escopo desta análise: baseada exclusivamente na US 1085 e suas Tasks, nas atas de reunião de levantamento, nos dois endpoints identificados e nos payloads/retornos efetivamente disponibilizados, além do histórico documentado da integração Sankhya × VokeNext (`voke.jar` , módulo 52). Tudo que não pôde ser comprovado está marcado como **pendência** ou **"Necessário validação com negócio"**. Nenhuma regra de negócio ou mapeamento Sankhya foi inventado.

1. Resumo Executivo

Problema identificado. A US 1085 registra que hoje existem informações do pedido/chamado que não estão sendo integradas adequadamente para o Sankhya, e que não havia um endpoint confiável para **consultar/listar** os chamados do VokeNext filtrando pelos campos corretos. Na análise dos endpoints, confirmou-se que o **Endpoint 1** (`serviceRequestIncident.load`) **não retorna dados de chamado** — retorna o HTML da tela e o contexto de sessão. O **Endpoint 2** (`.../servicerequestincident/search`) **retorna, sim, os dados do chamado em JSON estruturado**.

Objetivo da integração. Disponibilizar ao Sankhya a consulta dos chamados do VokeNext (com filtro), para trazer as informações do pedido/chamado que hoje faltam — em especial o vínculo de chamado pai → filho e os dados do produto/pedido embutidos no chamado.

Benefícios esperados. Visibilidade do status real do chamado no VokeNext a partir do Sankhya; base para reduzir a lacuna de dados apontada na US; insumo para rastreabilidade do fluxo de ativação. *(A quantificação de ganho depende de validação de negócio — pendência.)*

Impacto para Sankhya. Passa a ter uma fonte de consulta de chamados. O consumo desses dados por campo (gravação/atualização) depende de desenvolvimento e de validação de mapeamento — **não** está implementado hoje. *(Ver GAP Analysis.)*

Impacto para VokeNext. Nenhuma alteração estrutural aparente é exigida do lado VokeNext: o Endpoint 2 já expõe os dados. Ponto de atenção: o Endpoint 2 fornecido aponta para **ambiente de produção**, enquanto o Endpoint 1 é homologação (ver Riscos).

2. Inventário Completo dos Campos

2.1 Endpoint 1 — `serviceRequestIncident.load`

POST `https://vokenex-hom.4bizoxigen.com/4biz/serviceRequestIncident/serviceRequestIncident.load`

Parâmetros de entrada (body):

Campo	Tipo	Exemplo	Descrição Funcional	Obrigatório?
<code>object.id</code>	inteiro	6908	ID do chamado alvo (não filtrou o retorno no teste)	Necessário validação com negócio
<code>object.idContrato</code>	inteiro	6	Contrato	Necessário validação com negócio
<code>object.nomeServico</code>	string	"Atendimento e Suporte Cliente Locação"	Nome do serviço	Necessário validação com negócio

Retorno observado: documento HTML (tela AngularJS) + contexto de sessão. **Não retornou o chamado.** Campos estruturados presentes no retorno (contexto de sessão, não do chamado): `USER_LOGGED` (`idUsuario` , `login` , `email` , `grupos[]` , `colGrupos[]` , `contratos[]` , `colContratos[]` ...) e `LOWCODE_CONFIG` . Documentação detalhada em `reunioes/endpoint-serviceRequestIncident-load.md` .

Conclusão do Endpoint 1: não é fonte de dados de chamado. Serve a tela. Descartado como fonte de integração de dados.

2.2 Endpoint 2 — `servicerequestincident/search`

`https://vokenex.4bizoxygen.com/4biz/webmvc/servicerequestincident/search` — **retorna array JSON de chamados.**

Campo	Tipo	Exemplo	Descrição Funcional	Obrigatório?
<code>idSolicitacaoServico</code>	inteiro	1168	Identificador do chamado/solicitação de serviço	Necessário validação com negócio
<code>idStatus</code>	inteiro	1	Código do status do chamado	Necessário validação com negócio
<code>status</code>	string	"In Progress"	Descrição textual do status	Necessário validação com negócio
<code>idGrupoAtual</code>	inteiro	630	ID do grupo atual responsável	Necessário validação com negócio
<code>nomeGrupoAtual</code>	string	"Monitoramento de Chamado Pai"	Nome do grupo atual	Necessário validação com negócio
<code>siglaGrupo</code>	string	"MONI689"	Sigla do grupo atual	Necessário validação com negócio
<code>idTarefa</code>	inteiro	9264	ID da tarefa atual do fluxo	Necessário validação com negócio
<code>tarefa</code>	string	"Aguardando encerramento do chamado filho"	Nome da tarefa atual	Necessário validação com negócio

Campo	Tipo	Exemplo	Descrição Funcional	Obrigatório?
tipoServico	string	"N"	Tipo de serviço (código)	Necessário validação com negócio
nomeServico	string	"Ativação"	Nome do serviço	Necessário validação com negócio
nomeServicoRelacionado	string	"Solicitar Atendimento"	Serviço relacionado	Necessário validação com negócio
nomeTipoDemandaServico	string	"Requisição"	Tipo da demanda	Necessário validação com negócio
descricao	string	"Cod. Produto: 30645 - Descricao: DESKTOP DELL OPTIPLEX 7020... - Quantidade: 1 - Tipo Faturamento: Entrega - Imagem: 14.07-Padrão - Valor unitário: R\$ 105,00"	Texto livre com dados do produto/pedido concatenados	Necessário validação com negócio
contrato	string	"PAGUE MENOS"	Nome do contrato	Necessário validação com negócio
idContrato	inteiro	6	ID do contrato	Necessário validação com negócio
localidade	string	"Padrão"	Localidade	Necessário validação com negócio
idUnidade	inteiro	1	ID da unidade	Necessário validação com negócio
nomeUnidadeSolicitante	string	"Integração"	Unidade do solicitante	Necessário validação com negócio
nomeSolicitante	string	"Usuario Integração"	Nome do solicitante	Necessário validação com negócio
emailcontato	string	"usuario.integracao@run2biz.com"	E-mail de contato	Necessário validação com negócio
telefonecontato	string	" "	Telefone de contato (vazio no exemplo)	Necessário validação com negócio
origem	string	"Portal"	Origem do chamado	Necessário validação com negócio
idOrigem	inteiro	1	Código da origem	Necessário validação

Campo	Tipo	Exemplo	Descrição Funcional	Obrigatório?
				com negócio
<code>impacto</code>	string	"A"	Impacto (código)	Necessário validação com negócio
<code>urgencia</code>	string	"A"	Urgência (código)	Necessário validação com negócio
<code>sla</code>	string	"To be defined"	SLA	Necessário validação com negócio
<code>dataHoraInicio</code>	datetime	"2026-07-15 16:58:52 PM BRT"	Data/hora de início	Necessário validação com negócio
<code>dataHoraSolicitacao</code>	datetime	"2026-07-15 16:58:52 PM BRT"	Data/hora da solicitação	Necessário validação com negócio
<code>solicitanteVip</code>	boolean	false	Indicador de solicitante VIP	Necessário validação com negócio
<code>enderecoUnidade.idEndereco</code>	inteiro	6	ID do endereço da unidade	Necessário validação com negócio
<code>enderecoUnidade.enderecoStr</code>	string	" "	Endereço formatado (vazio no exemplo)	Necessário validação com negócio

Nota factual: o exemplo retornado é um **chamado pai** (grupo `MONI689` "Monitoramento de Chamado Pai", tarefa "Aguardando encerramento do chamado filho"), do serviço "Ativação", contrato PAGUE MENOS. Isso é coerente com o cenário pai → filho (904 → 905 → 906) discutido na reunião. **Não há**, neste payload, campo explícito de chamado-pai / chamado-filho — a relação não está evidente nos dados disponibilizados (**pendência**).

3. Consolidação dos Dados Disponíveis

- **Campos exclusivos do Endpoint 1:** apenas contexto de sessão (`USER_LOGGED` , `LOWCODE_CONFIG`) — **nenhum dado de chamado**. Não há campo de chamado exclusivo aproveitável.
- **Campos exclusivos do Endpoint 2:** todos os campos de chamado listados em 2.2 (o Endpoint 1 não retorna nenhum deles).
- **Campos existentes em ambos:** conceitualmente `idContrato` aparece nos dois (no E1 como parâmetro de entrada; no E2 como dado retornado). Não há sobreposição real de **dados de saída** entre os endpoints.
- **Possíveis redundâncias (dentro do Endpoint 2):** `dataHoraInicio` = `dataHoraSolicitacao` (mesmo valor no exemplo); pares código/descrição redundantes: `idStatus` / `status` , `idGrupoAtual` / `nomeGrupoAtual` / `siglaGrupo` , `idTarefa` / `tarefa` , `idOrigem` / `origem` , `idContrato` / `contrato` , `idUnidade` / `nomeUnidadeSolicitante` .
- **Possíveis conflitos de informação: ambiente divergente** — E1 = homologação (`vokenex-hom`), E2 = produção (`vokenex`). Consultar os dois como "mesma base" produziria dados de ambientes diferentes (risco

alto). Além disso, dados do produto/pedido só existem como **texto concatenado** em `descricao`, sem campos discretos — conflito potencial de parsing.

4. Mapeamento VokeNext → Sankhya

Campos Sankhya citados abaixo são **exclusivamente** os documentados no histórico da integração (`AD_TGFINTE`, `AD_TGFIINTE`, `TCSCON`, `TGFCAB`, `TGFITE`). Onde não há campo Sankhya documentado equivalente, registra-se como pendência. O mapeamento **não** foi validado com negócio.

Campo VokeNext	Campo Sankhya (documentado)	Origem Endpoint	Situação
<code>idSolicitacaoServico</code>	<code>AD_TGFINTE.IDCHAMADOITSM</code>	E2	Já Integrado (na escrita Sankhya → Voke) / consulta reversa: Não Integrado
<code>idContrato</code> / <code>contrato</code>	<code>TCSCON.NUMCONTRATO</code> (contrato)	E2	Necessário Validação de Negócio
<code>descricao</code> (Cod. Produto, Qtd, Valor unitário)	<code>TGFITE.CODPROD</code> / <code>TGFITE. QTDNEG</code> / <code>TGFITE.VLRUNIT</code> (itens do pedido)	E2	Necessário Desenvolvimento (parsing de string) + Validação de Negócio
<code>idStatus</code> / <code>status</code> / <code>tarefa</code>	<code>AD_TGFINTE.STATUS</code> / <code>STATUSPICKING</code>	E2	Necessário Validação de Negócio (semântica difere do <code>STATUSPICKING</code> interno)
<code>nomeServico</code> ("Ativação")	Fluxo de ativação (<code>AD_PLANROL</code> / pedido)	E2	Necessário Validação de Negócio
<code>siglaGrupo</code> / <code>nomeGrupoAtual</code>	(sem campo Sankhya documentado)	E2	Campo Não Encontrado
<code>nomeSolicitante</code> / <code>emailcontato</code> / <code>telefonecontato</code>	(sem campo Sankhya documentado)	E2	Necessário Validação de Negócio
<code>impacto</code> / <code>urgencia</code> / <code>sla</code>	(sem campo Sankhya documentado)	E2	Necessário Validação de Negócio
<code>idUnidade</code> / <code>nomeUnidadeSolicitante</code> / <code>localidade</code>	(sem campo Sankhya documentado)	E2	Necessário Validação de Negócio
<code>dataHoraInicio</code> / <code>dataHoraSolicitacao</code>	<code>AD_TGFINTE.DHINTEGRACAO</code> (aproximação)	E2	Necessário Validação de Negócio
relação chamado pai ↔ filho	(não há campo no payload nem campo Sankhya documentado)	—	Campo Não Encontrado

5. GAP Analysis

Informações que o VokeNext possui e o Sankhya ainda não recebe (via estes endpoints): status/tarefa atual do chamado, grupo atual, dados do solicitante/contato, impacto/urgência/SLA e o texto de produto em `descricao`. Hoje a integração documentada é **Sankhya → Voke**; o retorno desses dados **para** o Sankhya não está implementado.

Informações necessárias para atender a US que ainda não foram encontradas: - Um **campo explícito de vínculo pai → filho** — o payload não traz `idChamadoPai` / `idChamadoFilho` (a US pede o fluxo 904 → 905 → 906). **Pendência.** - O **campo de filtro** exato aceito pelo Endpoint 2 (parâmetros de query da busca) — não foi disponibilizado o request, apenas o retorno. **Pendência.** - **Autenticação** do Endpoint 2 (headers/token) — não disponibilizada. **Pendência.**

Informações existentes mas sem definição funcional: `tipoServico` ("N"), `idOrigem`, códigos de `impacto` / `urgencia` ("A"), `idTarefa`, `idStatus` — significados não documentados. **Necessário validação com negócio.**

Dependências de negócio para conclusão: definição de quais campos do chamado devem, de fato, ser gravados/atualizados no Sankhya e em qual tabela; regra de parsing do `descricao`; e confirmação do ambiente (produção vs homologação).

6. Proposta de Solução

(Com base exclusivamente nas evidências.)

- **Melhor fonte de dados: Endpoint 2** (`servicerequestincident/search`) — é o único que retorna os dados do chamado em JSON estruturado.
 - **Endpoint complementar: Endpoint 1** — **não** é fonte de dados; deve ser descartado para integração. Serve apenas como carga de tela.
 - **Dados suficientes para atender a US:** o Endpoint 2 fornece status, grupo, tarefa, contrato, solicitante e a descrição do produto — cobre boa parte da "dor" de visibilidade. **Insuficiente** para o vínculo pai → filho (não há campo) e para acionar filtros (request não disponibilizado).
 - **Dados que ainda precisam ser buscados:** parâmetros de filtro do Endpoint 2, campo de vínculo pai → filho, autenticação, e a semântica dos códigos.
 - **Riscos** (ver seção 8/Pontos de Atenção): ambiente de produção no Endpoint 2; dados de produto apenas como texto concatenado; ausência de contrato/paginação documentados.
-

7. Especificação Funcional para Projetos

Contexto. Evolução da integração Sankhya × VokeNext (fluxo de ativação, cliente Pague Menos), para trazer ao Sankhya informações do chamado hoje ausentes (US 1085).

Cenário Atual. Integração documentada é unidirecional Sankhya → Voke. A consulta de chamados carecia de endpoint confiável; o `serviceRequestIncident.load` não retorna dados.

Cenário Proposto. Consumir o Endpoint 2 (`search`) para obter os chamados em JSON e disponibilizar os campos relevantes no Sankhya (consulta e, mediante validação, gravação). Campos e tabela de destino a definir com negócio.

Dados Necessários. Campos do Endpoint 2 (seção 2.2); regra de extração do `descricao`; definição do filtro de consulta; autenticação.

Fluxo de Integração (proposto, macro). 1. Autenticar no VokeNext → 2. Chamar `search` com filtro (contrato/grupo/status — a confirmar) → 3. Receber array de chamados → 4. Transformar/validar → 5. Persistir/atualizar no Sankhya (destino a definir).

Dependências. Definição de negócio dos campos-alvo; parâmetros de filtro do endpoint; ambiente correto (homologação); autenticação.

Riscos. Endpoint 2 em produção; parsing frágil de `descricao`; ausência de vínculo pai → filho.

Pendências. Ver seção 9.

Critérios de Aceite (herdados/derivados da US 1085): 1. Endpoint de consulta identificado e documentado em ambiente de teste (sem uso de produção). 2. Campo de filtro que segmenta corretamente os chamados definido e validado. 3. Fluxo pai → filho compreendido e documentado. 4. Consulta retornando os chamados esperados. 5. Levantamento validado com a área de negócios.

8. Especificação Técnica para Desenvolvimento

Endpoint Consumido. `https://vokenex.4bizoxygen.com/4biz/webmvc/servicerequestincident/search` (**produção** — validar equivalente em homologação antes de desenvolver).

Método HTTP. Não disponibilizado explicitamente; o retorno é um array JSON. **Pendência** — validar (GET com query params ou POST com body de filtro).

Estrutura de Resposta. Array de objetos de chamado. Campos em 2.2. Objeto aninhado `enderecoUnidade`.

Campos Utilizados. A definir com negócio; candidatos diretos: `idSolicitacaoServico`, `idStatus / status`, `tarefa`, `idContrato / contrato`, `descricao`, `nomeServico`, `dataHoraInicio`.

Transformações Necessárias. - Parsing do `descricao` para extrair `Cod. Produto`, `Descricao`, `Quantidade`, `Tipo Faturamento`, `Imagem`, `Valor unitário` (formato R\$ 105,00 → decimal). **Regra a validar.** - Conversão de data "YYYY-MM-DD HH:mm:ss PM BRT" para o formato Sankhya.

Regras de Integração. Não definidas — dependem do mapeamento validado (seção 4). **Não inventar.**

Tratamento de Erros. Alinhar ao padrão existente do `voke.jar` (`IntegracaoLog` : `erroPedidos / erroEtapas / erroNotas / erroCMDB`), incluindo retentativa (hoje ausente no processo — melhoria conhecida).

Logs Recomendados. Registrar chamada, filtro usado, quantidade de chamados retornados, e falhas de parsing do `descricao`.

Pontos de Atenção. 1. **Ambiente:** Endpoint 2 aponta para **produção** — a reunião alertou explicitamente para não usar produção em teste. Confirmar host de homologação. 2. **descricao como texto:** dados de produto não são campos discretos — parsing frágil. 3. **Sem paginação/contrato documentados** no Endpoint 2 — validar volume e filtros. 4. **Autenticação não fornecida** para o Endpoint 2. 5. **Vínculo pai → filho ausente** no payload.

9. Perguntas em Aberto (a responder para concluir a solução)

1. Qual o **host de homologação** equivalente ao Endpoint 2 (o fornecido é produção)?
 2. Qual o **método HTTP** e os **parâmetros de filtro** aceitos pelo `search` (contrato? grupo? status? número)?
 3. Como se **autentica** no Endpoint 2 (token/headers)?
 4. Existe campo/endpoint que exponha explicitamente o **vínculo pai → filho** do chamado?
 5. Quais campos do chamado devem ser **efetivamente gravados/atualizados no Sankhya**, e em qual **tabela de destino**?
 6. Qual a **regra oficial de parsing** do campo `descricao` (produto, quantidade, valor, imagem)?
 7. Qual o significado dos **códigos** `tipoServico` ("N"), `idStatus`, `impacto / urgencia` ("A"), `idOrigem`, `idTarefa`?
 8. A consulta deve ser **pontual** (por chamado) ou **em lote** (listagem com filtro), e com qual periodicidade?
 9. Há **paginação** no retorno do `search` para grandes volumes?
 10. O consumo é **somente leitura** (visibilidade) ou também **escrita** de volta no fluxo?
-

Documento produzido a partir das evidências disponíveis no projeto. Itens marcados como pendência ou "Necessário validação com negócio" não devem ser tratados como definidos.